MINI CHESS AI

Innovative Assignment – 1
Foundations of Artificial Intelligence
Batch 20

1. Problem Formulation

The project develops a small chess engine that selects a move using depth-limited Minimax with Alpha-Beta pruning. A chess position is represented as an 8×8 board. The two players are White and Black, and the available actions are legal piece moves generated from the current board. The search explores possible future positions to select a move that maximizes the evaluation for the maximizing player and minimizes it for the opponent. The assignment requires material/positional evaluation, Alpha-Beta pruning, legal-move generation, and an analysis of search depth and pruning effect.

2. Approach

Board Representation: The board is stored as a list of eight rows. Uppercase pieces represent White and lowercase pieces represent Black. A dot represents an empty square.

Evaluation Function: The program uses material values: Pawn=1, Knight=3, Bishop=3, Rook=5, Queen=9 and King=100. White material contributes positively and Black material negatively. Therefore, a positive score favours White and a negative score favours Black.

Minimax: At each search level, the maximizing player chooses the highest evaluation while the minimizing player chooses the lowest evaluation. The search stops when the selected depth reaches zero or when no moves are available.

Alpha-Beta Pruning: Alpha stores the best guaranteed value for the maximizing side and beta stores the best guaranteed value for the minimizing side. When beta becomes less than or equal to alpha, remaining branches at that node are skipped because they cannot improve the final decision. The program counts both searched nodes and pruned branches.

3. Complexity Analysis

Let b be the branching factor and d be the search depth. A depth-limited Minimax search has approximately O(b^d) time complexity and O(d) recursive search depth, excluding storage for generated board states. Alpha-Beta pruning can reduce the number of explored nodes substantially; in favourable move ordering its theoretical best case approaches O(b^(d/2)), while the worst case remains O(b^d). The practical search limit in this implementation is depth 3.

4. Results and Comparison

The program was executed at search depths 1, 2 and 3. The output screenshots record the selected move, evaluation, number of nodes searched and branches pruned. The depth-2 run recorded 46 nodes searched and 11 branches pruned, with best move b8c6 and evaluation 0.0.

Search depth	Best move	Evaluation	Nodes searched	Branches pruned
1	See screenshot	See screenshot	See screenshot	See screenshot
2	b8c6	0.0	46	11
3	See screenshot	See screenshot	See screenshot	See screenshot

MINI CHESS AI — PROGRAM OUTPUT

Minimax + Alpha-Beta Pruning | Search Depth: 2

```
==============================
       MINI CHESS AI
==============================
Enter AI search depth (1-3): 2

Initial Board:

  a b c d e f g h
8 r n b q k b n r
7 p p p p p p p p
6 . . . . . . . .
5 . . . . . . . .
4 . . . . . . . .
3 . . . . . . . .
2 P P P P P P P P
1 R N B Q K B N R

Searching...

AI Best Move:
b8c6

Evaluation: 0.0
Nodes searched: 46
Branches pruned: 11
```

Figure 2: Mini Chess AI output at search depth 2

Interpretation

Increasing search depth allows the AI to examine more future positions and can improve decision quality, but it also increases computational work. The node and pruning counters provide direct evidence of this trade-off. Alpha-Beta pruning improves efficiency by eliminating branches that cannot affect the selected result.

5. Reflection

The main challenge was implementing move generation and connecting board transitions to recursive Minimax search. Another important practical issue was verifying the project through automated tests and correcting the Python source-file import setup. A future version could add full check/checkmate legality, castling, en-passant, promotion, stronger positional features, move ordering and a richer game interface.

Honesty note: AI assistance and online/reference material were used during development. The implementation and design choices should be understood and explained by the student during the viva.

6. Conclusion

The Mini Chess AI demonstrates adversarial search using depth-limited Minimax with Alpha-Beta pruning and a material-based evaluation function. The tested program executes successfully, and its search statistics provide measurable evidence for analysing depth and pruning.